Tensors in PyTorch

Rank, Shape, Dtype, Operations, Devices, Reshaping, and NumPy Interoperability

Abstract

A tensor is PyTorch's central data structure: a multidimensional array whose values are governed by a shape, data type, device, and storage layout. These notes develop that idea from scalars through image and video batches, then build a practical workflow for construction, reproducibility, arithmetic, broadcasting, reductions, linear algebra, mutation, copying, accelerator placement, reshaping, and NumPy conversion. Every major operation is paired with its shape contract, a manual calculation, runnable code, and common failure modes. Several easy-to-miss inaccuracies are corrected, including rank terminology, channel order, uninitialized memory, aliasing, accelerator timing, and shared NumPy storage.

Contents

1 Purpose and prerequisite vocabulary

Deep-learning systems repeatedly transform collections of numbers. Images contain pixel values, text is converted to token identifiers or embedding coordinates, model parameters contain weights and biases, and optimization produces gradient values. PyTorch represents all of these using tensors so that the same indexing, mathematical, differentiation, and device APIs can be applied consistently.

The following words must be separated:

Scalar A single number, such as 5.0.

Vector An ordered one-axis collection of numbers, such as $[0.12, -0.84, 0.33]$.

Matrix A two-axis rectangular collection organized by rows and columns.

Axis One independent index needed to locate a value. Axis numbers begin at zero in PyTorch.

Rank The number of axes. PyTorch exposes it as `tensor.ndim`.

Shape The length of every axis in order. A rank-two tensor with two rows and three columns has shape $[2, 3]$.

Element One stored value. `tensor.numel()` returns the total number of elements.

Dtype The numerical representation of each element, such as `torch.float32` or `torch.int64`.

Device The execution and storage location, such as CPU memory or an NVIDIA CUDA device.

Operation A rule that consumes one or more tensors and produces a result, possibly by mutating an existing tensor.

Definition

A *tensor* is a multidimensional array equipped with an operational contract: values, shape, dtype, device, and storage layout. A tensor is not defined by its printed values alone.

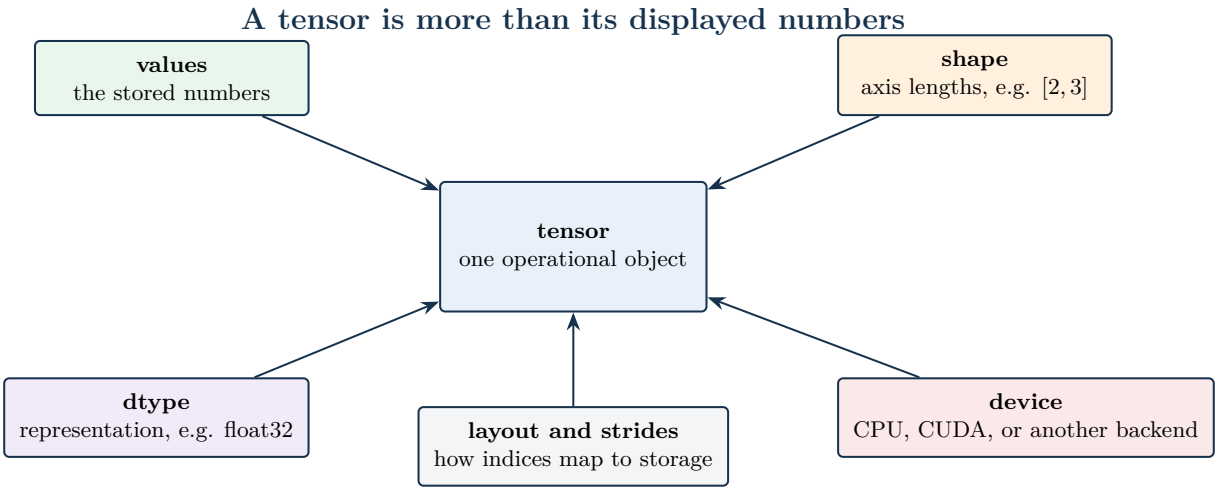

Figure 1: A tensor operation must satisfy the contracts associated with values, shape, dtype, device, and layout. Official reference.

1.1 Rank is not physical direction

Rank is sometimes described informally as the number of "directions" in which an object spreads. That picture becomes unreliable beyond three dimensions. The precise definition is simpler: rank is the number of indices required to select one element.

For a matrix X with shape $[2, 3]$, $X_{1,2}$ needs two indices and is therefore rank two. For a video batch V with shape $[B, T, C, H, W]$, $V_{b,t,c,h,w}$ needs five indices and is therefore rank five. Here B is batch size, T is frame count, C is channel count, H is height, and W is width.

Shape check

A scalar tensor has shape [ ] and rank zero. A one-element vector has shape [1] and rank one. Both contain one element, but their shapes and broadcasting behavior differ.

2 From scalar values to video batches

2.1 Rank zero: a scalar loss

A training objective normally reduces many prediction errors to one scalar loss. For one prediction $\hat{y} = 3.5$ and one target $y = 4.0$, a squared-error loss is

$$\mathcal{L} = (\hat{y} - y)^2 = (3.5 - 4.0)^2 = (-0.5)^2 = 0.25.$$

The symbol $\hat{y}$ denotes the predicted value, y denotes the known target, and $\mathcal{L}$ denotes the scalar loss. The result 0.25 measures squared discrepancy; it matters because automatic differentiation requires a well-defined objective from which parameter gradients can be computed.

2.2 Rank one: a word embedding vector

An embedding maps a discrete item to a dense numeric vector. A three-coordinate illustration is

$$e_{\text{word}} = [0.12, -0.84, 0.33] \in \mathbb{R}^3.$$

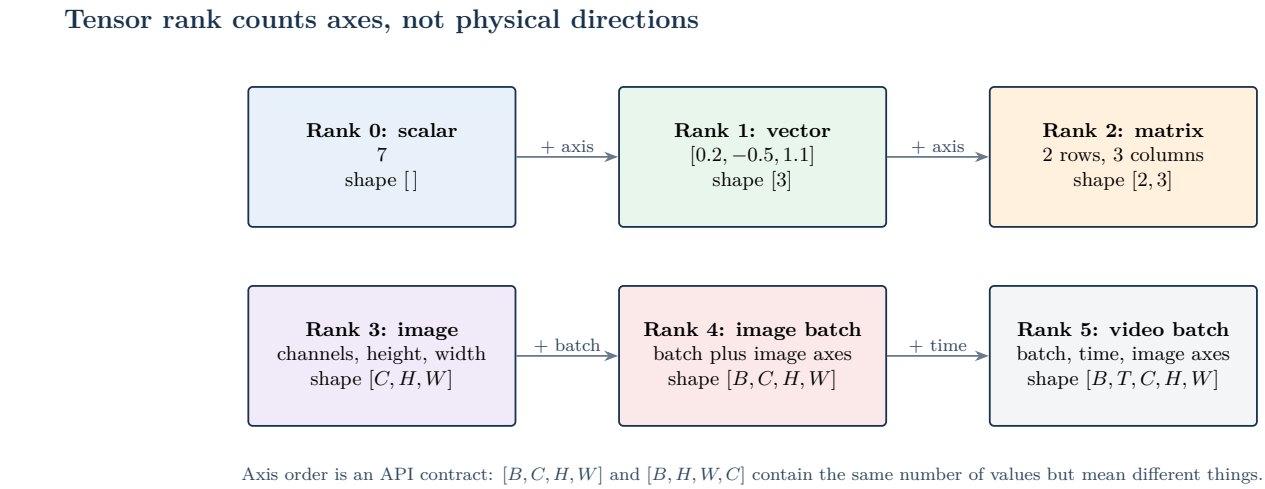

Figure 2: Tensor rank progresses from a scalar to vectors, matrices, images, image batches, and video batches by adding semantically named axes. Official reference.

The symbol e_{word} denotes an embedding for one word, and $\mathbb{R}^3$ means three real-valued coordinates. Systems such as Word2Vec and GloVe learn much longer vectors; each coordinate participates in comparisons and later neural transformations even though a coordinate rarely has a simple standalone label.

2.3 Rank two: a grayscale image matrix

A small grayscale image can be represented by

$$G = \begin{bmatrix} 0 & 255 & 128 \\ 34 & 90 & 180 \end{bmatrix}, \qquad G \in \mathbb{R}^{2\times 3}.$$

The row index selects vertical position, the column index selects horizontal position, and each entry is a pixel intensity. The shape is $[2, 3]$: two rows and three columns. In an eight-bit image, 0 is black and 255 is white, with intermediate values representing gray levels.

2.4 Rank three: one color image

A color image needs one axis for color channels in addition to height and width. A channel-last representation may have shape $[256, 256, 3]$, meaning height 256, width 256, and three red–green–blue channels. PyTorch vision models conventionally use channel-first shape $[3, 256, 256]$.

Pitfall

Shape values do not reveal axis meaning. Both $[256, 256, 3]$ and $[3, 256, 256]$ contain the same number of elements, but a model expecting channel-first input will misinterpret channel-last input. Use explicit names such as $[H, W, C]$ and $[C, H, W]$ in explanations and tests.

2.5 Rank four: a batch of color images

A batch adds a sample axis. A channel-last batch of 32 images, each 128×128 with three channels, has shape

$$[B, H, W, C] = [32, 128, 128, 3].$$

The symbol $B = 32$ counts images. A conventional PyTorch convolutional model usually expects $[B, C, H, W] = [32, 3, 128, 128]$. Batching permits one vectorized operation to process several samples and gives the optimizer a multi-sample estimate of the training objective.

2.6 Rank five: a batch of video clips

A batch of 10 clips, each containing 16 frames of size 64×64 with three channels, can use channel-last shape

$$[B, T, H, W, C] = [10, 16, 64, 64, 3].$$

The five axes identify clip, time, row, column, and color channel. A video API may instead require $[B, C, T, H, W]$ or $[B, T, C, H, W]$. The documentation for the consuming operation decides the valid order.

Key takeaway

Rank counts axes; shape records axis lengths; axis names record meaning. Correct deep-learning code needs all three.

3 Why tensors are useful in deep learning

Three capabilities make tensors central:

1. **Mathematical operations.** Addition, multiplication, dot products, matrix products, nonlinear functions, and reductions are implemented by optimized numerical kernels.
2. **A common representation.** Tabular features, token identifiers, embeddings, images, audio, video, weights, biases, activations, losses, and gradients can all be represented as tensors.
3. **Hardware-aware execution.** Supported operations can run on CPUs, GPUs, and other backends when the workload, dtype, and device placement are suitable.

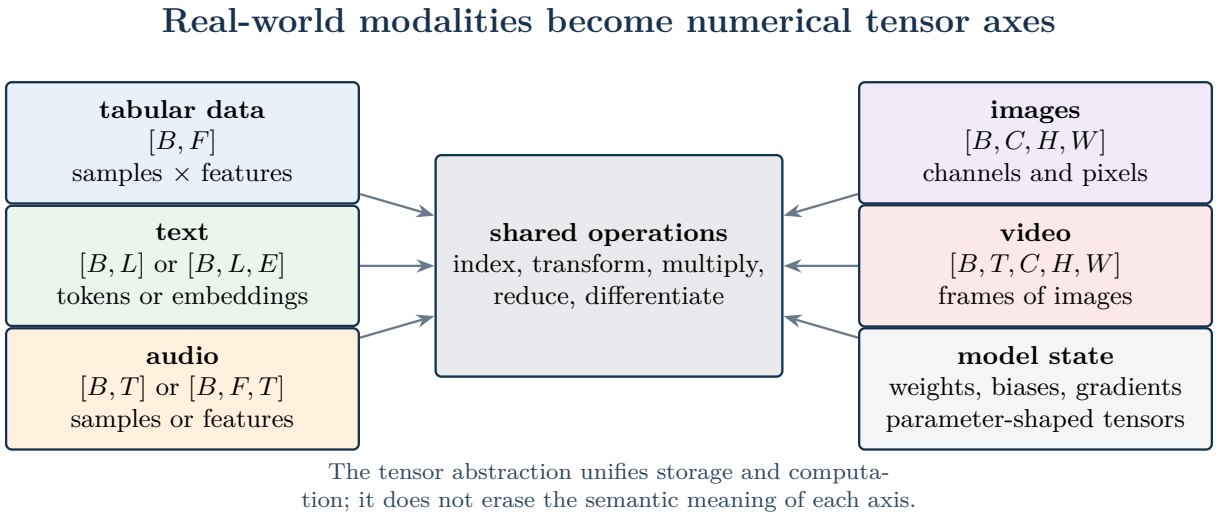

Figure 3: Different data modalities become tensors with different axis meanings while sharing a common family of operations. Official reference.

3.1 A neural layer as tensor mathematics

For a mini-batch, a dense transformation is

$$Y = XW + b.$$

The symbols and shapes are:

- $X \in \mathbb{R}^{B \times F}$: B input samples with F features each;

- $W \in \mathbb{R}^{F \times O}$: learnable weights from F inputs to O outputs;
- $b \in \mathbb{R}^{O}$: one learnable bias per output;
- $Y \in \mathbb{R}^{B \times O}$: transformed outputs.

The matrix product XW computes weighted sums, and broadcasting reuses b for each of the B rows. For one sample $X = [2, -1]$, two outputs, weights

$$W = \begin{bmatrix} 0.5 & 1 \\ 2 & -1 \end{bmatrix}, \qquad b = [0.1, 0.2],$$

the expanded calculation is

$$Y_1 = 2(0.5) + (-1)(2) + 0.1 = -0.9,$$
$$Y_2 = 2(1) + (-1)(-1) + 0.2 = 3.2.$$

Thus $Y = [-0.9, 3.2]$. Every number is produced by tensor operations; during training, gradients of the loss with respect to W and b are tensors with matching parameter shapes.

4 Runtime inspection and tensor creation

4.1 Inspect the environment before depending on it

A notebook or script should inspect its PyTorch build and available devices rather than assuming a GPU exists. A hosted runtime can restart when its accelerator type changes; all imports and variables must then be recreated.

```
import torch

print(torch.__version__)
if torch.cuda.is_available():
    print("GPU is available")
    print(torch.cuda.get_device_name(0))
else:
    print("GPU not available; using CPU")
```

Listing 1: Inspect the PyTorch build and optional CUDA device

A source environment reported PyTorch `2.5.1+cu121`; before accelerator selection it reported CPU use, and after a hosted-runtime restart it reported a Tesla T4. Those strings describe that environment only. A CUDA-enabled build can exist even when `torch.cuda.is_available()` is false.

4.2 Constructor families

```
import torch

uninitialized = torch.empty(2, 3)       # values are unspecified
zeros = torch.zeros(2, 3)
ones = torch.ones(2, 3)
filled = torch.full((3, 3), 5.0)
identity = torch.eye(5)

random_values = torch.rand(2, 3)        # uniform on [0, 1)
custom = torch.tensor([[1, 2, 3], [4, 5, 6]])
steps = torch.arange(0, 10, 2)          # [0, 2, 4, 6, 8]
points = torch.linspace(0, 10, 10)      # 10 evenly spaced values
```

Listing 2: Create tensors with common constructor families

The constructors serve different responsibilities:

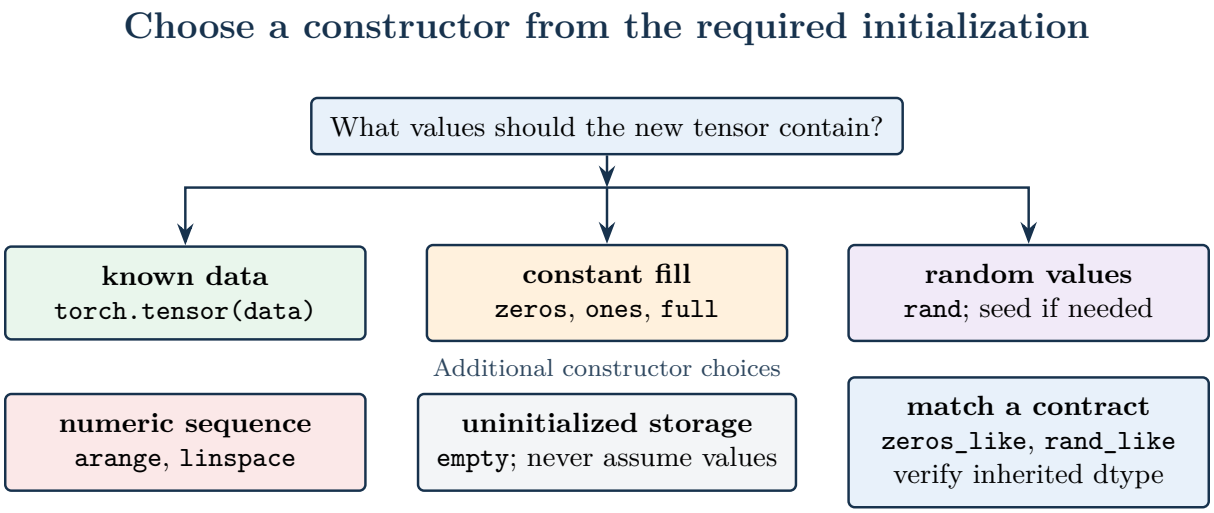

Figure 4: The correct constructor depends on whether values are known, constant, random, sequential, or intentionally uninitialized. Official reference.

`torch.empty` Allocates storage without initializing it. Printed values are whatever bit patterns occupy the reused memory and must never be treated as data.

`torch.zeros`, `torch.ones`, `torch.full` Initialize every element to a known constant. Zero initialization is often appropriate for biases or counters, but not universally for all learnable weights.

`torch.rand` Samples a uniform floating-point value from the half-open interval $[0, 1)$ for each position.

`torch.tensor` Copies or converts explicitly supplied Python data and infers a dtype unless one is specified.

`torch.arange` Uses a start, exclusive stop, and step. The sequence `arange(0, 10, 2)` stops before 10.

`torch.linspace` Includes both endpoints and chooses equal spacing. With n points from a to b, adjacent spacing is $(b - a)/(n - 1)$ when $n > 1$.

`torch.eye` Creates a rank-two identity matrix whose main diagonal is one and whose other entries are zero.

4.3 Reproducible random examples

A pseudo-random generator is deterministic once its state is fixed. Resetting a seed before the same operation reproduces values in the same compatible environment:

```
torch.manual_seed(100)
a = torch.rand(2, 3)

torch.manual_seed(100)
b = torch.rand(2, 3)

assert torch.equal(a, b)
# Current verified CPU run, first row of a:
# [0.1116643, 0.8158431, 0.2625620]
```

Listing 3: Reset a seed before two equivalent random draws

Pitfall

A seed is necessary but not sufficient for universal bit-for-bit reproducibility. Results can vary with PyTorch version, device backend, parallel algorithm, operation order, and nondeterministic kernels. Record the environment and consult PyTorch's reproducibility guidance when exact replay matters.

5 Shape, rank, dtype, and like-constructors

Let

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}.$$

This tensor has shape $[2, 3]$, rank 2, and $2 \cdot 3 = 6$ elements. The first shape entry counts rows; the second counts columns.

```
x = torch.tensor([[1, 2, 3], [4, 5, 6]])

print(x.shape)       # torch.Size([2, 3])
print(x.ndim)        # 2
print(x.numel())     # 6
print(x.dtype)       # torch.int64

u = torch.empty_like(x)   # same contract; unspecified values
z = torch.zeros_like(x)   # int64 zeros, shape [2, 3]
o = torch.ones_like(x)    # int64 ones, shape [2, 3]

# Random uniform values require a floating-point dtype.
r = torch.rand_like(x, dtype=torch.float32)
y = x.to(torch.float32)
```

Listing 4: Inspect shape and dtype and create shape-matched tensors

Like-constructors inherit important properties from a reference tensor. By default, `zeros_like(x)` and `ones_like(x)` copy shape, dtype, device, and layout where supported. `empty_like(x)` copies the contract but still leaves values uninitialized.

The integer tensor above cannot be passed to `torch.rand_like(x)` without a dtype override because a uniform random number in $[0, 1)$ cannot be represented meaningfully by the inherited `int64` dtype. The verified runtime raised `"check_uniform_bounds" not implemented for 'Long'`. Supplying `dtype=torch.float32` states the intended representation.

5.1 Important dtype families

Examples	Family	Typical responsibility
`torch.bool`	Boolean	Masks and comparison results.
`torch.int8, int32, int64`	Integer	Counts, indices, labels, and token identifiers; many indexing APIs require `int64`.
`torch.float16, bfloat16, float32, float64`	Floating point	Features, parameters, activations, losses, and numerical computation.
`torch.float8_e4m3fn and related variants`	Low-precision floating point	Specialized accelerator workflows; operator and backend support is narrower than for float32.
`torch.complex64, complex128`	Complex	Frequency-domain and other complex-valued calculations.

Converting dtype with `x.to(torch.float32)` returns a tensor in the requested representation. Conversion can round, truncate, overflow, or lose precision. For example, converting 2.9 to an integer does not preserve the fractional part.

6 Scalar arithmetic, broadcasting, and elementwise functions

6.1 Scalar operations

For

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad s = 2,$$

scalar addition computes $Y_{ij} = X_{ij} + s$ for every row i and column j. The compact form $Y = X + s$ therefore means

$$Y = \begin{bmatrix} 1+2 & 2+2 & 3+2 \\ 4+2 & 5+2 & 6+2 \end{bmatrix} = \begin{bmatrix} 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}.$$

The scalar has shape [ ] and is conceptually reused at all six positions. The result retains shape $[2, 3]$.

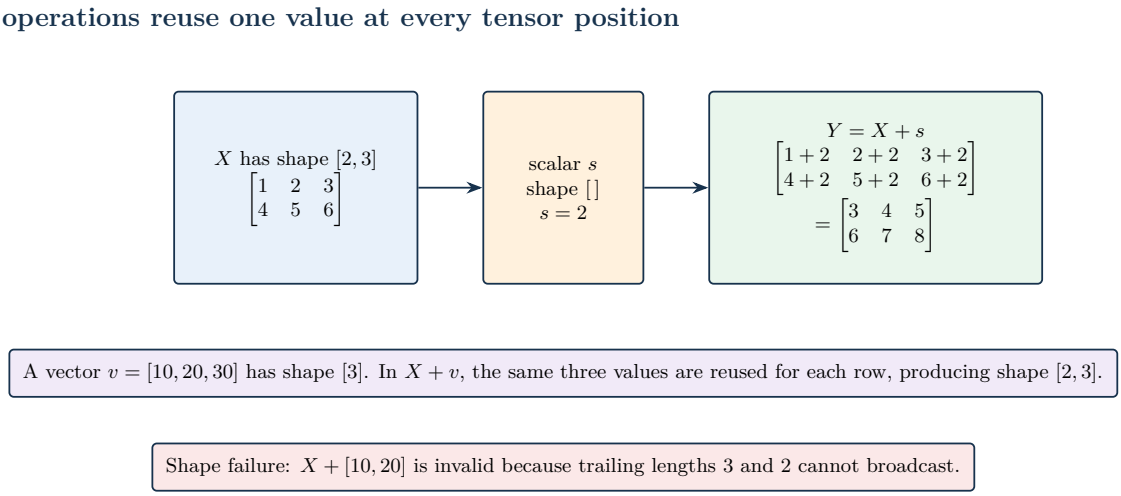

Figure 5: Scalar arithmetic is a simple broadcasting case in which one scalar value is reused at every tensor position. Official reference.

```
x = torch.tensor([[1., 2., 3.], [4., 5., 6.]])

plus = x + 2
minus = x - 2
product = x * 2
quotient = x / 2
square = x ** 2

v = torch.tensor([10., 20., 30.])
broadcast = x + v
assert broadcast.tolist() == [
    [11., 22., 33.],
    [14., 25., 36.],
]
```

Listing 5: Apply scalar arithmetic and explicit vector broadcasting

Broadcasting compares shapes from the final axis backward. Two lengths are compatible when they are equal or when one is 1; missing leading axes behave as if their length were 1. Thus $[2, 3] + [3]$ is valid because the final lengths match. By contrast, $[2, 3] + [2]$ fails because final lengths 3 and 2 are incompatible.

6.2 Elementwise binary and unary operations

Elementwise binary operations pair corresponding positions after broadcasting. If P and Q both have shape $[2, 3]$, then

$$(P \odot Q)_{ij} = P_{ij}Q_{ij},$$

where $\odot$ denotes elementwise multiplication. This is not matrix multiplication.

Unary functions transform each element independently. For

$$r = [1.9, 2.3, 3.7, 4.4],$$

rounding gives $[2, 2, 4, 4]$, ceiling gives $[2, 3, 4, 5]$, and floor gives $[1, 2, 3, 4]$. Clamping to $[2, 3]$ maps values below 2 to 2 and values above 3 to 3:

$$\text{clamp}(r, 2, 3) = [2, 2.3, 3, 3].$$

The operation matters when values must obey a known valid range. It can also hide upstream numerical problems if used without a principled reason.

```
p = torch.tensor([1., -2., 3., -4.])
r = torch.tensor([1.9, 2.3, 3.7, 4.4])

absolute = torch.abs(p)             # [1, 2, 3, 4]
negative = torch.neg(p)             # [-1, 2, -3, 4]
rounded = torch.round(r)            # [2, 2, 4, 4]
upper = torch.ceil(r)               # [2, 3, 4, 5]
lower = torch.floor(r)              # [1, 2, 3, 4]
clipped = torch.clamp(r, 2., 3.)    # [2, 2.3, 3, 3]

left = torch.tensor([5, 4])
right = torch.tensor([2, 6])
mask = left > right                 # [True, False]
```

Listing 6: Use unary transforms, clipping, and comparisons

Floor division `//`, remainder `%`, and exponentiation `**` are also elementwise. Their behavior depends on dtype and sign; a test should include negative operands when negative values are possible.

6.3 Logarithm, exponential, square root, sigmoid, softmax, and ReLU

Several common functions have domain or dtype requirements:

- $\log(x)$ requires $x > 0$ for a finite real result.
- $\sqrt{x}$ requires $x \geq 0$ for a real result.
- $\exp(x) = e^x$ can overflow for large positive floating-point inputs.
- $\text{ReLU}(x) = \max(0, x)$ replaces negative values with zero.
- $\sigma(x) = 1/(1 + e^{-x})$ maps a real value to the open interval $(0, 1)$.

For a vector $z \in \mathbb{R}^K$, softmax along its class axis is

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

Here z_i is the score for class i, K is the number of classes, and the denominator sums exponentials across the selected axis. The result is nonnegative and sums to one along that axis, so it can be interpreted as a normalized class distribution.

For $z = [1, 2, 3]$, the verified result is approximately

$$[0.09003, 0.24473, 0.66524].$$

The third entry is largest because the third input score is largest. Stable library implementations internally avoid unnecessary overflow; manually computing raw exponentials is less robust.

```
k = torch.tensor([1., 2., 3.])

logs = torch.log(k)
exponentials = torch.exp(k)
roots = torch.sqrt(k)
probabilities = torch.sigmoid(k)
classes = torch.softmax(k, dim=0)
activated = torch.relu(torch.tensor([-2., 0., 3.]))
```

Listing 7: Apply special functions to floating-point tensors

The `dim` argument defines which axis is normalized. Integer inputs to `mean`, `softmax`, and many transcendental functions must be converted to an appropriate floating-point dtype rather than relying on accidental coercion.

7 Reductions and statistics

A reduction combines several elements and therefore changes shape. Consider again

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \text{with shape } [2, 3].$$

Reducing all elements gives

$$\operatorname{sum}(X) = 1 + 2 + 3 + 4 + 5 + 6 = 21,$$

a scalar with shape $[\,]$. Reducing `dim=0` removes the row axis and keeps one result per column:

$$\operatorname{sum}(X, \text{dim} = 0) = [1 + 4, 2 + 5, 3 + 6] = [5, 7, 9].$$

Reducing `dim=1` removes the column axis and keeps one result per row:

$$\operatorname{sum}(X, \text{dim} = 1) = [1 + 2 + 3, 4 + 5 + 6] = [6, 15].$$

```
x = torch.tensor([[1., 2., 3.], [4., 5., 6.]])

sum_all = x.sum()           # tensor(21.)
sum_columns = x.sum(0)     # tensor([5., 7., 9.])
sum_rows = x.sum(1)        # tensor([6., 15.])
mean_all = x.mean()         # tensor(3.5)
mean_columns = x.mean(0)   # tensor([2.5, 3.5, 4.5])
median = x.median()         # tensor(3.)
minimum = x.min()           # tensor(1.)
maximum = x.max()           # tensor(6.)
product = x.prod()          # tensor(720.)
argmax = x.argmax()         # tensor(5), flat index of value 6
argmin = x.argmin()         # tensor(0), flat index of value 1
```

Listing 8: Reduce a matrix over all elements or a selected axis

An unseeded constructor example used `torch.randint(size=(2, 3), low=0, high=10)`. One captured run produced

$$\begin{bmatrix} 5 & 5 & 8 \\ 8 & 5 & 0 \end{bmatrix},$$

whose total is 31 and whose row sums are $[18, 13]$. Re-executing the random-construction cell can replace this matrix, so later outputs need not remain arithmetically consistent with an earlier screenshot unless

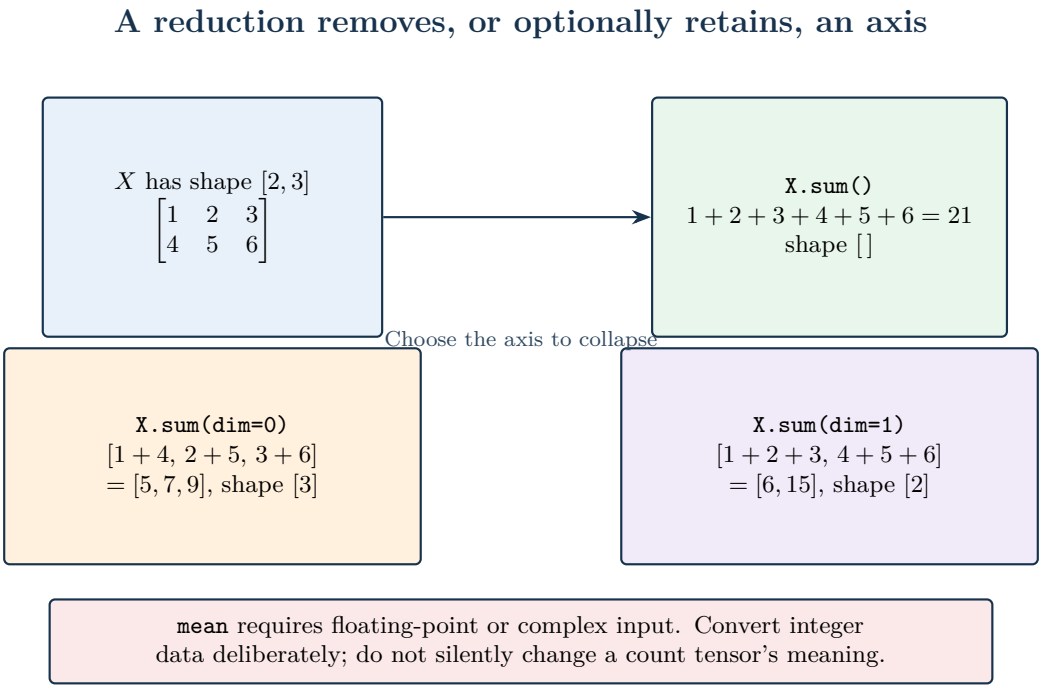

Figure 6: Reduction axes determine which positions are combined and which axis remains in the result. Official reference.

the value is fixed or the seed is reset. The deterministic matrix above prevents that state-dependent ambiguity.

`argmax()` and `argmin()` without a `dim` argument flatten conceptually and return one flat index. With a dimension, they return indices along that dimension. An index is not the same as the extreme value.

For values $x_1, \ldots, x_N$ with mean $\bar{x}$, population variance is

$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2,$$

and population standard deviation is $\sigma = \sqrt{\sigma^2}$. The symbol N is the number of values, $\bar{x}$ is their arithmetic mean, and each squared difference measures deviation from the mean. Variance has squared units; standard deviation returns to the original units. PyTorch's default `torch.var` and `torch.std` use a correction appropriate to sample estimation; set `correction=0` when the population definition is intended.

Pitfall

`torch.mean(torch.tensor([1, 2, 3]))` fails because the input is integer-valued. Decide whether a floating-point mean is mathematically meaningful, then convert explicitly with `.float()` or construct the tensor with a floating dtype.

8 Elementwise arithmetic and linear algebra

Let

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \in \mathbb{R}^{2\times 3}, \qquad W = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \in \mathbb{R}^{3\times 2}.$$

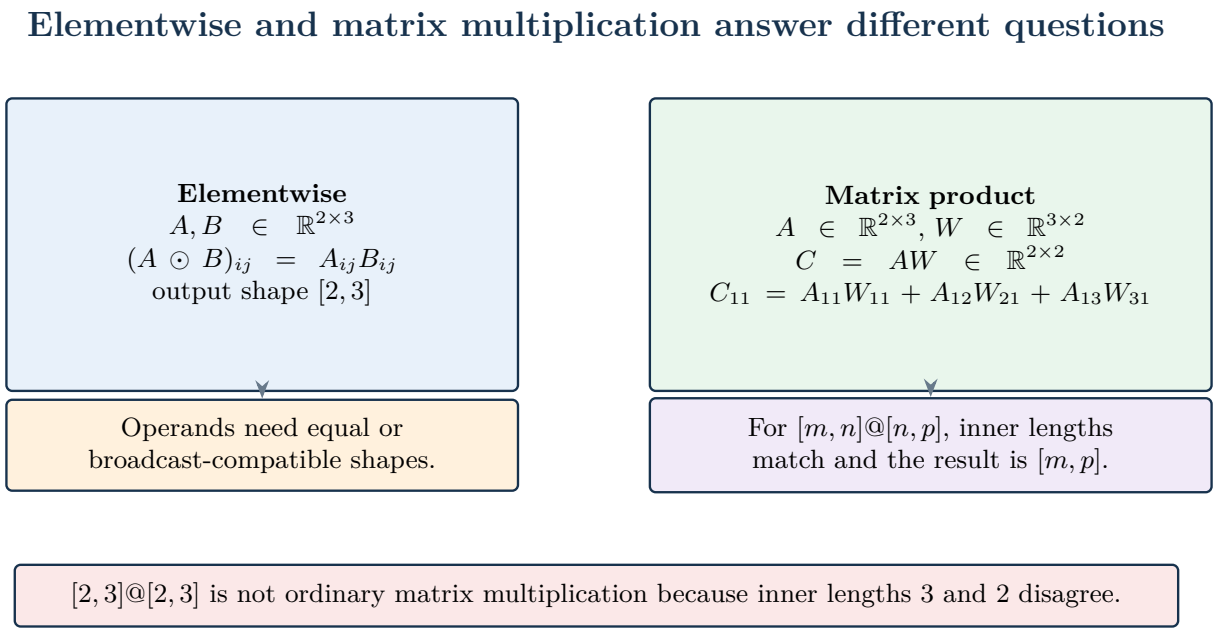

Figure 7: Elementwise multiplication preserves a broadcasted shape, whereas matrix multiplication contracts matching inner dimensions. Official reference.

The product $C = AW$ has shape $[2, 2]$ because the matching inner length 3 is contracted. Every output entry is a row–column dot product:

$$\begin{aligned} C_{11} &= 1(1) + 2(3) + 3(5) = 22, \\ C_{12} &= 1(2) + 2(4) + 3(6) = 28, \\ C_{21} &= 4(1) + 5(3) + 6(5) = 49, \\ C_{22} &= 4(2) + 5(4) + 6(6) = 64. \end{aligned}$$

Therefore

$$C = \begin{bmatrix} 22 & 28 \\ 49 & 64 \end{bmatrix}.$$

This operation matters because dense neural layers, attention mechanisms, and many feature transformations are built from matrix products.

```
a = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
w = torch.tensor([[1., 2.], [3., 4.], [5., 6.]])

c = torch.matmul(a, w)  # or a @ w; shape [2, 2]
d = torch.dot(torch.tensor([1., 2.]),
              torch.tensor([3., 4.]))  # 1*3 + 2*4 = 11
transposed = a.T        # shape [3, 2]

m = torch.tensor([[4., 7.], [2., 6.]])
determinant = torch.linalg.det(m)  # approximately 10
inverse = torch.linalg.inv(m)
```

Listing 9: Perform matrix, vector, transpose, determinant, and inverse operations

Current-practice note

Earlier examples often call `torch.det(m)` and `torch.inverse(m)`. The `torch.linalg.det` and `torch.linalg.inv` names make the linear-algebra responsibility explicit and belong to the current `torch.linalg` interface. Both pairs express the same demonstrated operations for this matrix; new code should follow the current official documentation.

Transposition swaps two matrix axes: $(A^\top)_{ij} = A_{ji}$. A determinant is defined for a square matrix and summarizes properties including invertibility. For a 2×2 matrix

$$M = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad \det(M) = ad - bc.$$

Here a, b, c, d are matrix entries; a nonzero determinant is required for a unique inverse. For numerical computation, solving a system with `torch.linalg.solve` is often preferable to explicitly constructing an inverse.

9 Out-of-place operations, in-place mutation, aliases, and clones

An out-of-place expression such as `c = a + b` allocates or selects storage for a result without changing `a` or `b`. Many PyTorch methods ending in an underscore, such as `add_` and `relu_`, mutate the receiving tensor. The underscore is a naming convention for available in-place variants; it does not imply that every function has one.

```
a = torch.tensor([[1., 2.], [3., 4.]])
b = torch.tensor([[10., 20.], [30., 40.]])

c = a + b
assert torch.equal(a, torch.tensor([[1., 2.], [3., 4.]]))

a.add_(b)
assert torch.equal(a, torch.tensor([[11., 22.], [33., 44.]]))

activation = torch.tensor([-2., 0., 3.])
activation.relu_()
assert activation.tolist() == [0., 0., 3.]
```

Listing 10: Contrast out-of-place and in-place addition

In-place mutation can reduce temporary allocation, but it changes program state and can invalidate values needed by automatic differentiation. A memory optimization is correct only when ownership and gradient requirements permit mutation.

9.1 Assignment is not a copy

The statement `b = a` binds a second Python name to the same tensor object. Mutating through either name is visible through both. By contrast, `a.clone()` creates a tensor with copied values and distinct tensor storage.

```
a = torch.tensor([1, 2, 3])
b = a
c = a.clone()

assert id(a) == id(b)
assert id(a) != id(c)
assert a.data_ptr() != c.data_ptr()

a[0] = 99
assert b.tolist() == [99, 2, 3]
assert c.tolist() == [1, 2, 3]
```

Listing 11: Prove aliasing and independent cloning through mutation

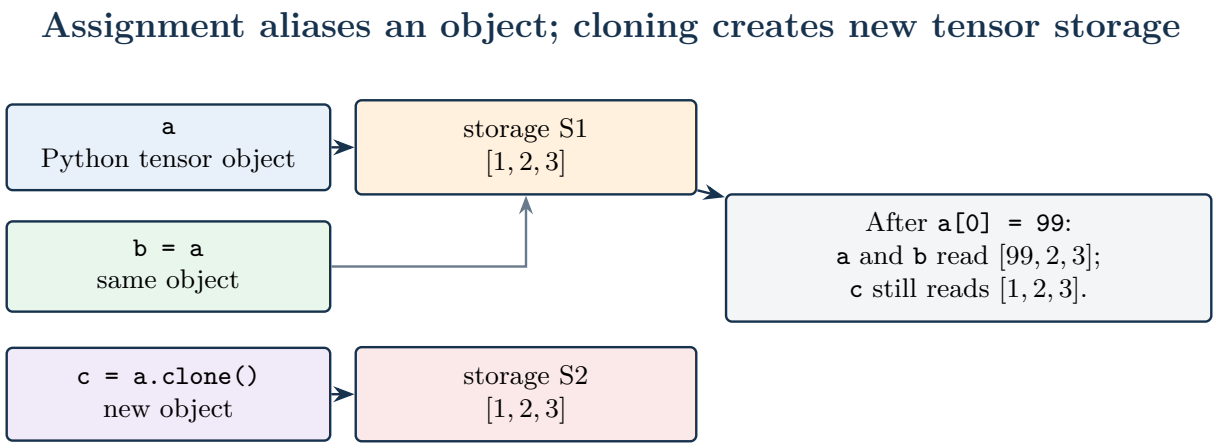

Figure 8: Assignment aliases one tensor object and storage, whereas cloning produces independently mutable storage. Official reference.

Pitfall

Python's `id()` identifies the lifetime identity of a Python object; it is not a general description of tensor storage. Different tensor objects can still share storage through views, slicing, or NumPy conversion. Use mutation tests and documented alias behavior, and call `clone()` when independent tensor storage is required.

10 Tensor operations on CPU and GPU

A device determines where storage lives and where compatible kernels execute. Merely having a GPU in the machine does not move tensors automatically. Operands used by one ordinary operation must be on compatible devices.

10.1 Select a device and move tensors deliberately

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

created_there = torch.rand(2, 3, device=device)
created_on_cpu = torch.rand(2, 3)
moved = created_on_cpu.to(device)
result = moved + 5

assert created_there.device == device
assert moved.device == device
```

Listing 12: Create and move tensors with an explicit device policy

`.to(device)` returns a tensor on the requested device; store the returned value. A CPU tensor and CUDA tensor cannot ordinarily be added directly. Moving a tensor also costs time and bandwidth, so repeatedly transferring small tensors can erase any compute advantage.

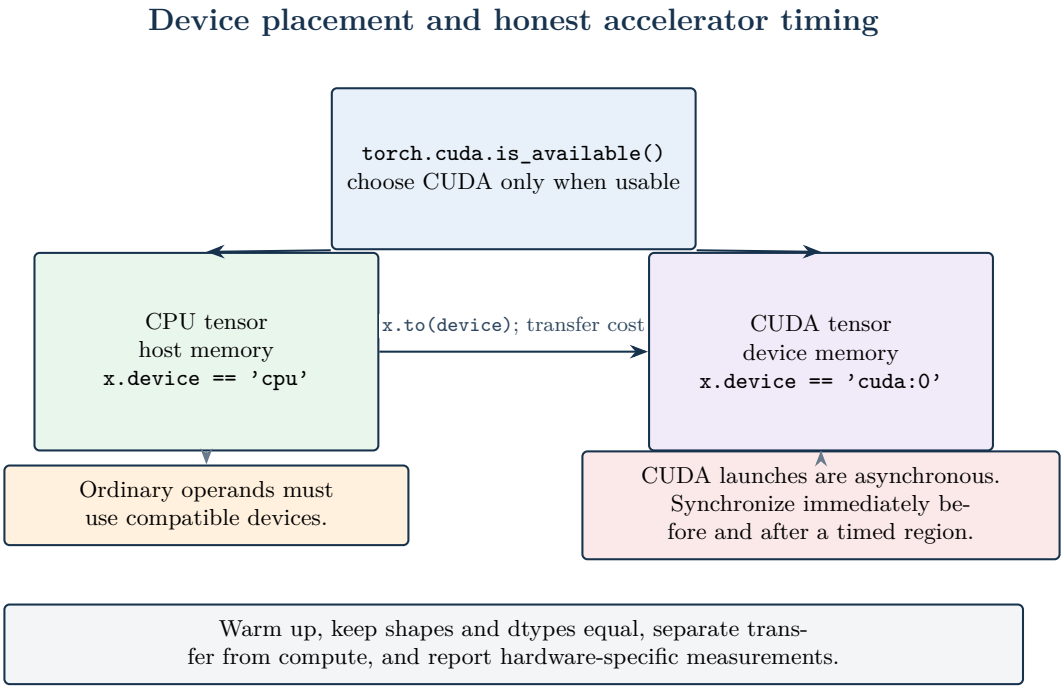

Figure 9: Device-aware code checks availability, controls transfers, keeps operands together, and synchronizes asynchronous accelerator work before timing. Official reference.

10.2 Interpret performance comparisons honestly

A source demonstration multiplied two 10,000 × 10,000 matrices on a CPU and a Tesla T4 and printed 17.7451 seconds on CPU, 0.5724 seconds on GPU, and a ratio of approximately 30.9985. Those values are an observation from one hosted session, not a PyTorch guarantee.

One float32 matrix of that shape contains 100,000,000 elements. At four bytes per element, its raw payload is

$$100{,}000{,}000 \times 4 = 400{,}000{,}000 \text{ bytes} \approx 381.47 \text{ MiB}.$$

The element count and four-byte representation define this payload; MiB divides by 2^{20}. Two inputs plus one result require about 1.12 GiB before allocator workspace and other process memory. Such a benchmark can fail on a smaller machine.

CUDA launches are asynchronous with respect to the CPU. Correct elapsed-time measurement must synchronize around the timed region:

```python
import time
import torch

n = 1024
cpu_a = torch.rand(n, n)
cpu_b = torch.rand(n, n)
start = time.perf_counter()
cpu_c = cpu_a @ cpu_b
cpu_seconds = time.perf_counter() - start

if torch.cuda.is_available():
    device = torch.device("cuda")
    gpu_a, gpu_b = cpu_a.to(device), cpu_b.to(device)
    _ = gpu_a @ gpu_b               # warm-up
    torch.cuda.synchronize()
    start = time.perf_counter()
    gpu_c = gpu_a @ gpu_b
    torch.cuda.synchronize()
    gpu_seconds = time.perf_counter() - start
```

Listing 13: Time matrix multiplication without confusing launch time and completion time

A fair comparison keeps shapes and dtypes equal, warms up both paths appropriately, separates transfer from compute when relevant, repeats measurements, and reports the processor, PyTorch build, precision, and statistic. The isolated verification environment for these notes was CPU-only; a safe 512×512 CPU multiplication completed and produced shape $[512, 512]$, while no GPU time was fabricated.

11 Reshaping and reordering axes

Shape transformations answer different questions:

`reshape` Regroups the same number of elements into new axis lengths.

`flatten` Collapses a selected range of axes, often all axes, into one.

`permute` Reorders axes without changing their lengths.

`unsqueeze` Inserts a length-one axis at a chosen position.

`squeeze` Removes a chosen length-one axis, or all singleton axes when no dimension is specified.

11.1 The element-count invariant

If an original tensor has shape $[d_1, d_2, \ldots, d_k]$, its element count is

$$N = \prod_{i=1}^{k} d_i.$$

The symbol d_i is the length of axis i, k is rank, and N is total element count. A valid reshape to $[e_1, \ldots, e_m]$ must satisfy

$$\prod_{i=1}^{k} d_i = \prod_{j=1}^{m} e_j.$$

The equality matters because reshaping reorganizes existing elements; it does not invent or delete them.

For a tensor of shape $[2, 3, 4]$, $N = 2 \cdot 3 \cdot 4 = 24$. Shapes $[4, 6]$, $[2, 12]$, and $[24]$ are valid because each product is 24. Shape $[5, 5]$ is invalid because it requests 25 elements.

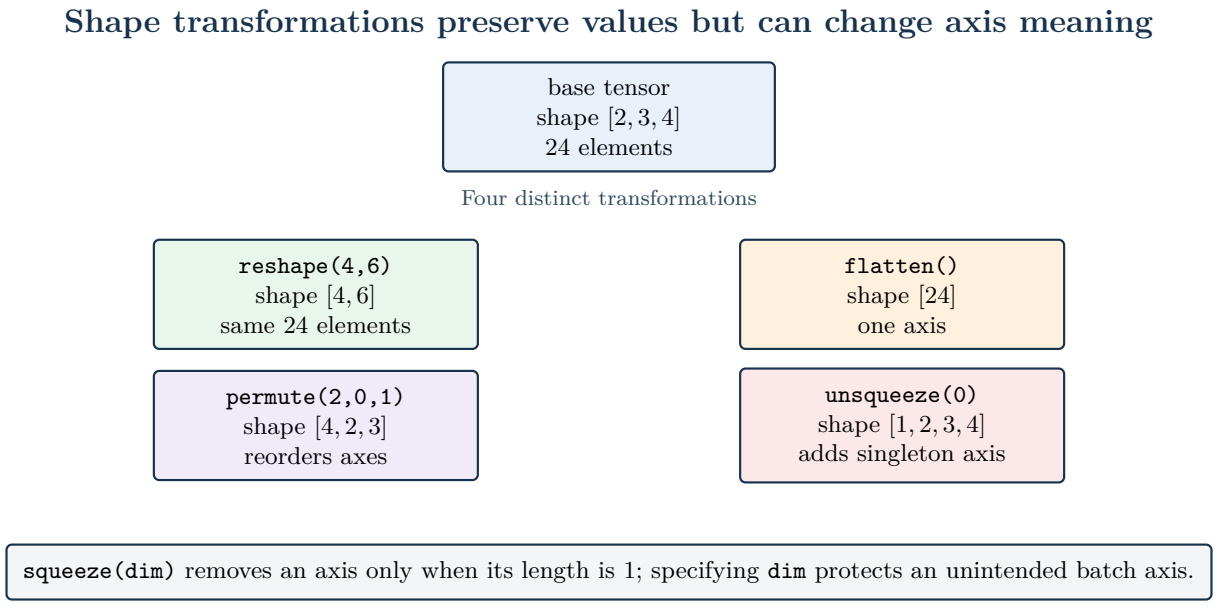

Figure 10: Reshape, flatten, permute, unsqueeze, and squeeze preserve values while changing grouping, order, or rank. Official reference.

```
base = torch.arange(24).reshape(2, 3, 4)

matrix = base.reshape(4, 6)        # shape [4, 6]
vector = base.flatten()            # shape [24]
reordered = base.permute(2, 0, 1) # shape [4, 2, 3]

image_hwc = torch.zeros(224, 224, 3)
batch_hwc = image_hwc.unsqueeze(0) # shape [1, 224, 224, 3]
image_again = batch_hwc.squeeze(0)  # shape [224, 224, 3]

# Convert HWC to the channel-first batch contract used by many models.
batch_chw = image_hwc.permute(2, 0, 1).unsqueeze(0)
assert batch_chw.shape == (1, 3, 224, 224)
```

Listing 14: Reshape, flatten, permute, and manage singleton axes

The demonstrated shape cases were concrete: a $[4, 4]$ tensor of ones became $[2, 2, 2, 2]$ through `reshape`; flattening produced a length-16 vector; a random $[2, 3, 4]$ tensor became $[4, 2, 3]$ with `permute(2,0,1)` and $[4, 3, 2]$ with `permute(2,1,0)`; a $[226, 226, 3]$ image-shaped tensor became $[1, 226, 226, 3]$ after `unsqueeze(0)`; and squeezing dimension zero of $[1, 20]$ produced $[20]$. The runnable example above uses the common 224×224 image size and additionally converts channel-last data to the channel-first batch contract. The invariant is axis meaning, not a particular pixel size.

Pitfall

Calling `squeeze()` without `dim` removes every singleton axis. If a batch happens to contain one item, this can delete the batch axis and make downstream behavior depend on batch size. Prefer `squeeze(dim)` when only one known axis should be removed.

12 NumPy and PyTorch interoperability

NumPy arrays and CPU tensors can share underlying memory. This makes conversion efficient but introduces aliasing. A mutation through one object may be visible through the other.

```
import numpy as np
import torch

a = torch.tensor([1, 2, 3])
b = a.numpy()               # array([1, 2, 3]); shared CPU storage
assert type(b) is np.ndarray
b[0] = 10
assert a.tolist() == [10, 2, 3]

c = np.array([1, 2, 3])
d = torch.from_numpy(c)     # tensor([1, 2, 3]); shared storage
c[0] = 40
assert d.tolist() == [40, 2, 3]

independent_array = a.numpy().copy()
independent_tensor = torch.tensor(c)  # copies data by default
```

Listing 15: Convert between CPU tensors and NumPy arrays and expose shared storage

A tensor requiring gradients or residing on a non-CPU device needs an explicit boundary before ordinary NumPy conversion:

```
array = tensor.detach().cpu().numpy()
```

`detach()` removes the result from gradient tracking, `cpu()` ensures NumPy-compatible device placement, and `numpy()` exposes the CPU values. This sequence should be used only when leaving the differentiable PyTorch computation intentionally.

Pitfall

Conversion does not always imply copying. Use `.copy()` for an independent NumPy array and `clone()` or `torch.tensor(array)` when independent tensor storage is required. Also verify dtype: a NumPy `float64` array typically produces a `torch.float64` tensor, which may not match a float32 model.

13 Failure modes and a diagnostic workflow

1. **Confusing rank with element count.** A scalar [ ] and vector [1] both contain one element but have different ranks.
2. **Ignoring axis meaning.** Record shapes with names such as $[B, C, H, W]$, not only numbers.
3. **Treating `empty` output as data.** Initialize before reading.
4. **Assuming one seed controls every source of nondeterminism.** Record versions and deterministic-algorithm settings where replay matters.
5. **Inheriting an incompatible dtype.** Random, mean, softmax, logarithm, and gradient operations usually need floating-point or complex inputs.
6. **Using silent broadcasting unintentionally.** Assert expected operand and result shapes, especially when batch size can equal feature count by coincidence.
7. **Confusing `*` and `@`.** The former is elementwise; the latter follows matrix-product contraction rules.

8. **Reducing the wrong axis.** Test with a tiny matrix whose row and column reductions are visibly different.
9. **Using `argmax` as if it returned a value.** It returns an index; use indexing if the value is also required.
10. **Mutating a tensor needed by autograd.** Avoid in-place operations until ownership and gradient behavior are understood.
11. **Assuming assignment copies.** `b = a` aliases; use `clone()` for independent tensor storage.
12. **Assuming different Python objects never share storage.** Views and NumPy interoperability can alias memory.
13. **Mixing devices.** Move model state, inputs, and newly created constants to compatible devices.
14. **Timing CUDA without synchronization.** Measure completed work, not only kernel-launch latency.
15. **Ignoring benchmark memory.** Estimate input, output, and workspace allocation before choosing matrix sizes.
16. **Reshaping to a different element count.** Compare products of old and new shapes.
17. **Using `permute` as if it were `reshape`.** Permutation changes axis order; reshape changes grouping.
18. **Removing every singleton axis accidentally.** Pass a specific dimension to `squeeze`.
19. **Assuming NumPy conversion copies.** Mutate a tiny test or create an explicit copy.

A practical debugging sequence is:

1. print or assert `shape`, `ndim`, `dtype`, and `device`;
2. label each axis with its meaning;
3. reduce the example to small deterministic values;
4. expand one output element manually;
5. distinguish out-of-place results, views, aliases, and clones;
6. check finite values with `torch.isfinite`;
7. only then scale the operation or move it to an accelerator.

14 Key takeaways and review questions

Key takeaways

- A tensor's values, shape, dtype, device, and layout jointly define its operational contract.
- Rank counts axes; shape lists their lengths; named axes preserve semantic meaning.
- Scalars, embeddings, images, image batches, and video batches correspond naturally to ranks zero through five.
- Constructor choice determines initialization. `empty` never provides meaningful initial values.
- Broadcasting, reductions, and matrix multiplication each have distinct shape rules that should be checked manually on a small example.
- In-place mutation, assignment aliases, cloned storage, and shared NumPy memory represent different state behaviors.
- Accelerator speed depends on workload, transfer, synchronization, precision, hardware, and measurement method.
- Reshape changes grouping, permute changes axis order, and unsqueeze or squeeze changes rank.

Review questions

1. What is the difference among rank, shape, and element count?
2. Why do scalar shape [ ] and vector shape [1] behave differently?
3. Give channel-last and channel-first shapes for one RGB image and a batch of RGB images.
4. What does each axis mean in $[10, 16, 64, 64, 3]$?
5. Why are loss, weights, biases, activations, and gradients all suitable tensor values?
6. Why must `torch.empty` output be initialized before use?
7. What does `torch.manual_seed` guarantee, and what does it not guarantee?
8. Why does `torch.rand_like` fail when it inherits `int64`?
9. Expand every element of $X + 2$ for $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$.
10. Explain why shapes $[2, 3]$ and $[3]$ broadcast but $[2, 3]$ and $[2]$ do not.
11. What shapes result from reducing a $[2, 3]$ matrix along dimensions zero and one?
12. Distinguish `x * y`, `x @ y`, and `torch.dot(x, y)`.
13. Manually compute the first element of the matrix product used in these notes.
14. Why can an in-place operation interfere with automatic differentiation?
15. What happens after `b = a; a[0] = 99`, and how does `clone()` change the result?
16. Why is `id()` not a complete tensor-storage diagnostic?
17. What must be synchronized when timing CUDA operations, and why?
18. Estimate raw memory for one float32 matrix of shape [10,000, 10,000].
19. Why can $[2, 3, 4]$ reshape to $[4, 6]$ but not $[5, 5]$?
20. Convert one $[224, 224, 3]$ image to a channel-first batch shape $[1, 3, 224, 224]$.

21. Why can changing a NumPy array change a tensor created with `torch.from_numpy`?